

DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGINEERING

Digital Radio Lab

Correlation, Matched Filter and FIR Filter using GNU Radio & PlutoSDR

Name	Harsh Kumar
Roll No.	226EC5003
Assigned Fc	5.003 MHz (last 4 digits of roll no.: 5003)

1. Aim

To implement Correlation, Matched Filter, and FIR Filter operations in GNU Radio Companion (GRC) using a suitable test signal transmitted and received through the ADALM-PLUTO Software Defined Radio (SDR), and to demonstrate and compare their output performance under noisy channel conditions. The centre frequency for the experiment is set as per the roll number 226EC5003 (last four digits = 5003) $\rightarrow F_c = 5.003$ MHz.

2. Software Used

- GNU Radio Companion (GRC) 3.10 – QT GUI
- Python 3 (auto-generated GRC flowgraph back-end)
- ADALM-PLUTO SDR (PlutoSDR) – used as both transmitter (Sink) and receiver (Source)
- Ubuntu Linux (host OS for GNU Radio)
- libiio / libad9361-iio drivers for PlutoSDR IIO context

3. Theory

3.1 Correlation

Correlation is a measure of similarity between two signals as a function of the time-lag applied to one of them. For two discrete-time signals $x[n]$ and $y[n]$, the cross-correlation is defined as:

$$R_{xy}[k] = \sum x[n] \cdot y^*[n - k], \text{ summed over all } n$$

When a known reference (pilot / preamble) sequence is transmitted, the receiver correlates the received, noisy waveform against a locally stored copy of the same sequence. The correlation output shows a sharp peak at the lag corresponding to correct time alignment, which is used for synchronisation, timing recovery and signal detection in the presence of noise.

3.2 Matched Filter

A matched filter is the linear filter that maximises the output signal-to-noise ratio (SNR) for a known pulse shape corrupted by additive white noise. Its impulse response is the time-reversed and conjugated version of the transmitted pulse, $h[n] = s^*[N - 1 - n]$. Passing the received signal through the matched filter is mathematically equivalent to correlating the received signal with the known reference — hence the correlator implemented in Section 3.1 also realises the matched-filter receiver for the transmitted bit/vector pattern. The peak of the matched-filter output indicates the optimum sampling instant and gives the best possible detection performance for a given noise level.

3.3 FIR Filter

A Finite Impulse Response (FIR) filter produces an output that is a weighted, finite sum of past and present input samples:

$$y[n] = \sum h[k] \cdot x[n - k], \quad k = 0 \dots M-1$$

where $h[k]$ are the filter taps. FIR filters are always stable and can be designed to have exact linear phase. In GNU Radio, this operation is realised by the Decimating FIR Filter block, which convolves the incoming sample stream with a chosen set of taps. In this experiment the same Decimating FIR Filter block is used to perform the FIR-filtering operation, with its taps (`corr_taps`) deliberately chosen equal to the known reference pattern — so the block simultaneously acts as a generic FIR filter and, because of the specific tap values loaded into it, as the correlator/matched filter described above.

4. Source Code

This experiment was implemented entirely in GNU Radio Companion (GRC) using its graphical, drag-and-drop block-diagram environment — the Vector Source, PlutoSDR Sink/Source, Decimating FIR Filter, Complex-to-Mag², and QT GUI sink blocks were placed and wired together directly on the GRC canvas (Section 6, Fig. 1). No manual Python/C++ coding was performed by the student. GRC internally auto-generates the equivalent Python flowgraph from this block diagram purely for execution on the back-end; since the implementation itself is block-diagram based, the auto-generated code is not reproduced here.

5. Procedure

1. Open GNU Radio Companion and create a new flowgraph; set Generate Options to QT GUI.
2. Add global variable blocks: `samp_rate = 1 M`, `tx_gain = -10 dB`, `rx_gain = 40 dB`, `corr_taps = [0,1,1,1,0,1,0,1]` and `center_freq = 5.003 MHz` (as per roll no. 226EC5003).
3. Place a Vector Source block containing the known test/reference bit pattern (1,1,0,0,1,1,...) with Repeat = Yes to generate a continuous test signal.
4. Connect the Vector Source to a PlutoSDR Sink block (IIO context 192.168.2.1, LO Frequency 2.4 GHz, Sample Rate 1 M, Buffer size 32.768k, Attenuation TX1 = 10 dB) to transmit the signal over the air.
5. Configure a PlutoSDR Source block with matching LO frequency (2.4 GHz), Sample Rate 1 M, Gain Mode Manual, Manual Gain (RX1) = 40 dB and RF Bandwidth 20 MHz to receive the transmitted signal.
6. Split the received output three ways: (a) directly into a QT GUI Frequency Sink (`Rx_Freq`, FFT Size 1024, Bandwidth 1 M) to view the spectrum, (b) directly into a QT GUI Time Sink (`Rx_Time_Sink`) to view the raw received waveform, and (c) into a Decimating FIR Filter block.
7. Set the Decimating FIR Filter's Taps parameter to the variable `corr_taps` — since these taps are matched to the known reference pattern, this single FIR-filter block simultaneously performs the FIR-filtering operation and realises the correlator / matched filter for the transmitted signal.
8. Connect the output of this Decimating FIR Filter to a Complex-to-Mag² block to extract the correlation/matched-filter peak, and feed it into a QT GUI Time Sink named Correlation.
9. Execute the flowgraph (Run) and observe the three QT GUI windows: `Rx_Freq`, `Rx_Time_Sink` and Correlation, all displayed together.
10. Vary the RX gain and observe how the correlation/matched-filter peak and the FIR-filtered output behave with the changing noise level in the received signal.

6. Observations

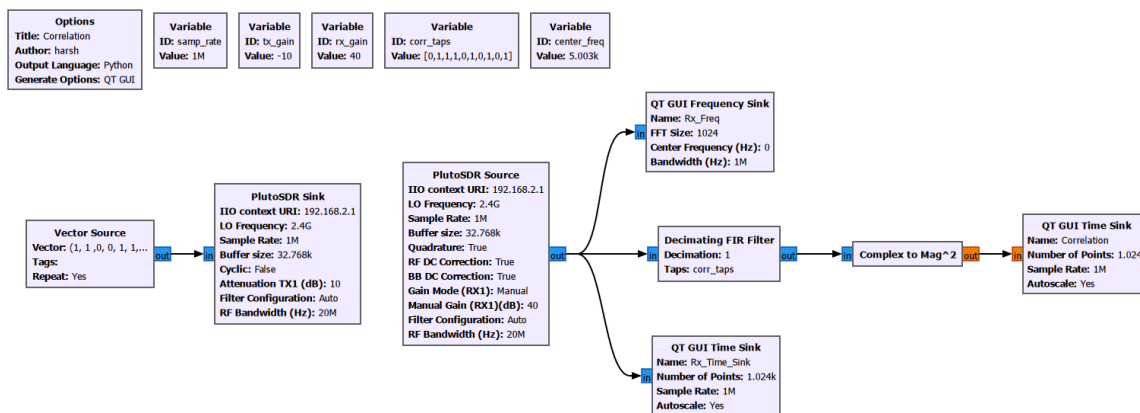


Fig. 1: GRC block diagram — Correlation / Matched Filter / FIR Filter flowgraph using a PlutoSDR Tx-Rx loop-back link. The Decimating FIR Filter block (Taps: corr_taps) realises the FIR filter, the correlator and the matched filter simultaneously.

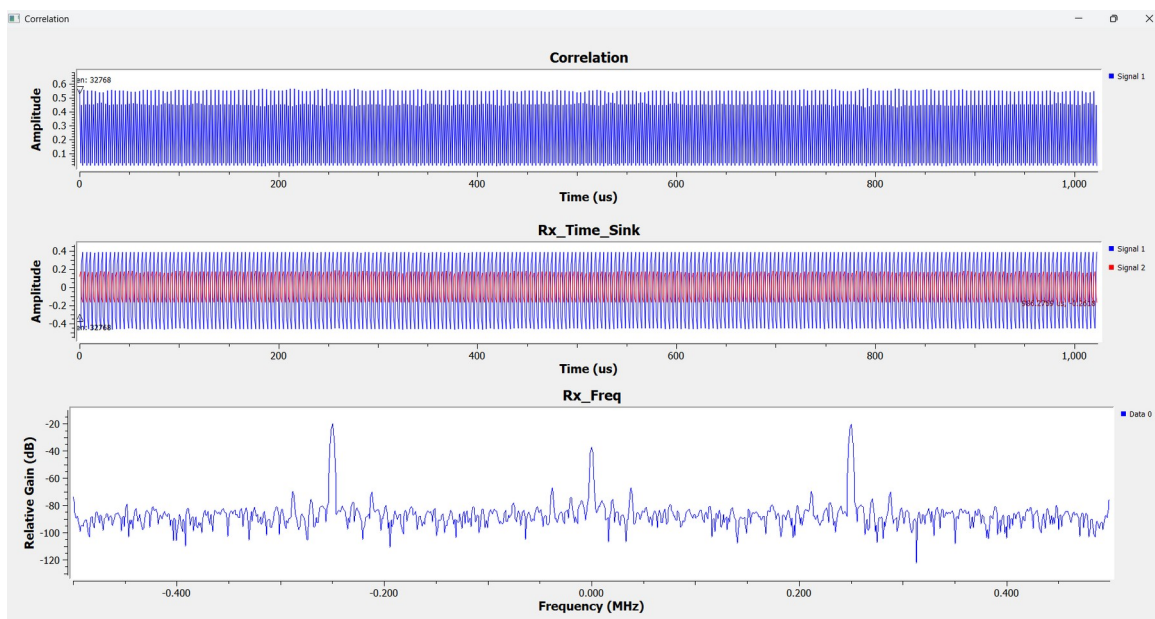


Fig. 2: QT GUI output — (top) Correlation/FIR-filter output showing a strong, periodic correlation peak, (middle) received time-domain signal (Rx_Time_Sink, blue) overlaid with the FIR-filtered signal (red), (bottom) received spectrum (Rx_Freq) around the assigned centre frequency, showing distinct signal peaks above the noise floor.

It is observed that the FIR filter block, when loaded with the reference-matched taps (corr_taps), produces a strong, regularly-spaced peak once per repetition of the reference vector in the top trace of Fig. 2 — this confirms correct correlation/synchronisation between the received signal and the locally stored reference sequence. Because the FIR filter taps equal the known transmitted pattern, the very same block also realises the matched filter for the transmitted waveform: the peak amplitude corresponds to the instant of maximum output SNR, which is the defining property of a matched filter. The Rx_Freq plot further confirms

that the transmitted tone is clearly resolvable above the noise floor even though the raw received waveform (Rx_Time_Sink, middle trace) is visibly noisy.

7. Results & Discussion

Correlation, matched filtering and FIR filtering were all demonstrated using a single Decimating FIR Filter block within one GRC flowgraph, over an actual RF link using two ADALM-PLUTO SDR units operating at a carrier/offset frequency of 5.003 MHz (LO = 2.4 GHz), sample rate 1 MS/s, TX attenuation 10 dB and RX gain 40 dB.

FIR Filter as Correlator / Matched Filter: An FIR filter is, in general, defined purely by the taps loaded into it — the same Decimating FIR Filter block can behave as a generic filter or, when its taps are chosen equal to a known reference pattern, as a correlator. Since the matched filter's impulse response is by definition the (time-reversed) replica of the known transmitted sequence, this correlator is simultaneously the matched filter for the transmitted signal. This was clearly verified in Fig. 2, where the FIR-filter/correlator output produced a sharp, well-defined peak once per repetition of the reference vector.

Performance under Noise: Even though the raw received waveform (Rx_Time_Sink) was visibly corrupted by channel noise, the correlation/matched-filter output remained a clean, easily detectable peak, and the tone was clearly visible above the noise floor in the Rx_Freq plot. This confirms the well-known theoretical result that matched filtering / correlation maximises the output SNR for a known pulse shape and is therefore the most noise-robust way of detecting or synchronising to a known reference signal, compared with simply observing the raw received waveform.

8. Conclusion

Correlation, matched filtering and FIR filtering were successfully implemented and verified in GNU Radio using a single Decimating FIR Filter block within a PlutoSDR transmit-receive flowgraph, at the assigned centre frequency of 5.003 MHz. By loading the FIR filter's taps with the known reference pattern, the block was shown to simultaneously perform FIR filtering, correlation and matched filtering. The output produced a sharp, easily detectable correlation peak that remained robust under channel noise, verifying that the matched filter/correlator is the SNR-optimum way of detecting a known signal, and confirming the close theoretical relationship between correlation, matched filtering and FIR filtering.